

02、python-SDK 介绍

一、SDK 程序

本项目是一个用于控制高擎机器人电机的 SDK，支持通过 CAN 总线和串口与电机控制器通信。SDK 提供了完整的 C++库以及 Python 绑定，可以方便地集成到各种控制系统中。

1、程序框架

代码块

```
1  hightorque_robot/
2  |—— CMakeLists.txt           # CMake构建配置
3  |—— include/                 # C++头文件
4  |   |—— hardware/           # 硬件相关类
5  |   |—— crc/                # CRC校验
6  |   |—— serial_*.hpp        # 串口相关
7  |—— src/                    # C++源文件
8  |   |—— hardware/           # 硬件相关类
9  |   |—— crc/                # CRC校验
10 |   |—— serial_*.cpp         # 串口相关
11 |—— example/                # C++示例程序
12 |   |—— motor_run.cpp        # 电机控制示例
13 |—— python/                 # Python绑定和示例
14 |   |—— pybind_wrapper.cpp   # pybind11包装器
15 |   |—— *.py                 # Python示例
16 |—— robot_param/            # 机器人配置文件
17 |   |—— robot_config.yaml     # 配置文件启动文件
18 |   |—— *.yaml               # 机器人配置文件示例
19 |—— msg/                    # LCM消息定义
20 |—— lib/                     # 依赖包
```

2、使用说明

1. 用户主要使用部分

- Python 控制程序
 - 目录： `python/`
 - 示例： `example_motor_control.py` （可复制修改为自己的控制程序）
 - 可以在该目录编写新的 `.py` 控制脚本，调用底层接口实现电机控制。
- 机器人配置文件

- 目录: `robot_param/`
- 用途: 根据所用电机型号与数量修改, 确保参数与硬件匹配。

2. 底层实现 (无需修改)

- C++ 核心代码
 - 目录: `src/`、`include/`
 - 功能: 封装了电机控制、通信、数据处理等核心逻辑。
 - 说明: 如需了解控制逻辑可查看此处源码, 正常使用不必改动。
- 依赖与消息定义
 - 目录: `lib/`、`msg/`
 - 功能: 第三方库与 LCM 消息结构体定义。

3. 电机控制函数

- 所有底层电机控制函数都实现于 `src/` 目录下, 并通过 `python/pybind_wrapper.cpp` 暴露给 Python 层。
- 用户在 Python 脚本中直接调用, 例如:

代码块

```
1  motor.position(1.0)           # 位置控制
2  rb.motor_send_2()
3
4  motor.velocity(2.0)           # 速度控制
5  rb.motor_send_2()
6
7  motor.pos_vel_MAXtqe(...)     # 位置+速度+力矩控制
8  rb.motor_send_2()
```

- 如果需要了解具体实现, 可在 `src/` 中搜索对应函数名。

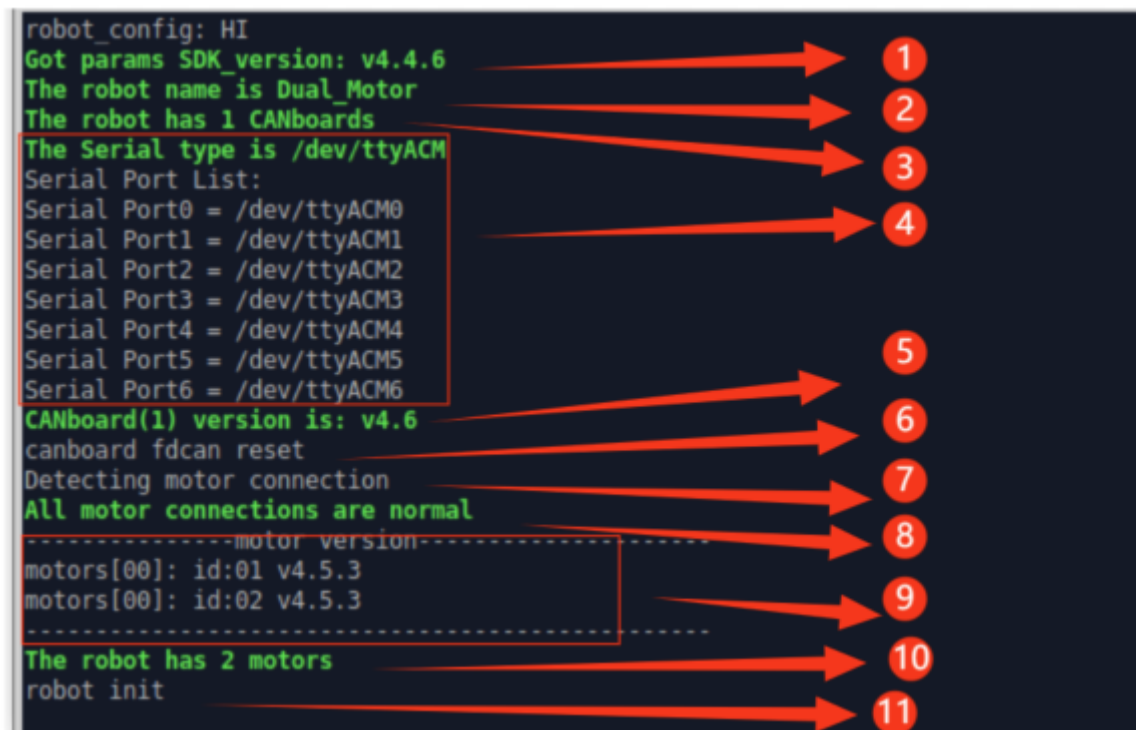
3、程序启动流程

1、创建机器人实例

代码块

```
1  robot = hr.Robot()
```

```
robot_config: HI
Got params SDK_version: v4.4.6
The robot name is Dual_Motor
The robot has 1 CANboards
The Serial type is /dev/ttyACM
Serial Port List:
Serial Port0 = /dev/ttyACM0
Serial Port1 = /dev/ttyACM1
Serial Port2 = /dev/ttyACM2
Serial Port3 = /dev/ttyACM3
Serial Port4 = /dev/ttyACM4
Serial Port5 = /dev/ttyACM5
Serial Port6 = /dev/ttyACM6
CANboard(1) version is: v4.6
canboard fdcan reset
Detecting motor connection
All motor connections are normal
-----motor version-----
motors[00]: id:01 v4.5.3
motors[00]: id:02 v4.5.3
-----
The robot has 2 motors
robot init
```



robot初始化示例

调用 `hr.Robot()` 中的初始化程序，依次获取并打印以下信息：

1. SDK 版本信息
2. 机器人名称
3. 通讯板数量
4. 可用串口信息
5. 通讯板固件版本
6. fdcan 重置（需 SDK 版本 v4.1.0 及以上，通讯板固件版本 v4.1 及以上）
7. 开始检测电机连接
8. 所有电机连接均正常
9. 电机版本信息（需 SDK、通讯板固件、电机固件均为 v4 版本）
10. 配置的电机数量（统计在配置文件（yaml 文件）中配置的电机数量）
11. 初始化完成提示

2、启动通信

代码块

```
1 robot.lcm_enable()
```

- 开启 LCM 通信，用于与机器人及电机进行实时数据交换。

运行控制程序

- 进入编写的控制模式。
- 执行对应的电机控制与状态采集逻辑。

```
LCM init success
Found 2 motors
Motor 0: ID=1, Type=MotorType.m4438_30, Name=L_low_foot
Motor 1: ID=2, Type=MotorType.m4438_30, Name=L_up_foot
Starting motor control loop...
Press Ctrl+C to stop...
Motor 1: mode=0, fault=0x00, pos=0.358142, vel=0.001571, tor=0.000000
Motor 2: mode=0, fault=0x00, pos=0.363168, vel=0.001571, tor=0.000000
Motor 1: mode=0, fault=0x00, pos=0.358142, vel=-0.003142, tor=0.000000
Motor 2: mode=0, fault=0x00, pos=0.363168, vel=0.000000, tor=0.000000
Motor 1: mode=10, fault=0x00, pos=0.373850, vel=0.513650, tor=0.331128
Motor 2: mode=10, fault=0x00, pos=0.382018, vel=0.645597, tor=0.057816
Motor 1: mode=10, fault=0x00, pos=0.402752, vel=0.333009, tor=0.299592
Motor 2: mode=10, fault=0x00, pos=0.437938, vel=0.574911, tor=0.120888
Motor 1: mode=10, fault=0x00, pos=0.431655, vel=0.376991, tor=0.189216
Motor 2: mode=10, fault=0x00, pos=0.437938, vel=0.574911, tor=0.120888
Motor 1: mode=10, fault=0x00, pos=0.459301, vel=0.534071, tor=0.283824
Motor 2: mode=10, fault=0x00, pos=0.465584, vel=0.615752, tor=0.094608
Motor 1: mode=10, fault=0x00, pos=0.487575, vel=0.593761, tor=0.194472
Motor 2: mode=10, fault=0x00, pos=0.492602, vel=0.373850, tor=0.094608
Motor 1: mode=10, fault=0x00, pos=0.515221, vel=0.344004, tor=0.168192
Motor 2: mode=10, fault=0x00, pos=0.521504, vel=0.666018, tor=0.078840
Motor 1: mode=10, fault=0x00, pos=0.543496, vel=0.535642, tor=0.141912
Motor 2: mode=10, fault=0x00, pos=0.549150, vel=0.362854, tor=0.178704
```

打印信息为：

- ① 电机连接数量与电机型号
- ② 退出功能信息
- ③ 读取的电机状态信息

4、简单使用示例

代码块

```
1 import hightorque_robot_py as hr
2
3 # 创建机器人实例
4 robot = hr.Robot()
5 robot.lcm_enable() # 启用LCM通信 (可选)
6
7 # 获取电机列表
8 print(f"发现 {len(robot.Motors)} 个电机")
9
10 # 控制电机
11 for motor in robot.Motors:
```

```

12     # 位置-速度-最大力矩控制
13     motor.pos_vel_MAXtqe(position=1.0, velocity=0.5, max_torque=10.0)
14
15     # 发送控制命令
16     robot.motor_send_2()
17
18     # 读取电机状态
19     for motor in robot.Motors:
20         state = motor.get_current_motor_state()
21         print(f"电机 {state.ID}: 位置={state.position:.3f}, 速度=
           {state.velocity:.3f}")
22
23     # 停止所有电机
24     robot.set_stop()
25     robot.motor_send_2()

```

该例程为基本使用框架：

- 创建机器人对象
- 启用 LCM 通信（可选）
- 进行控制电机（向缓冲写入电机控制指令）
- 发送控制命令
- 读取电机状态（从缓冲读取数据，电机状态信息的解析在子线程中进行）
- 停止所有电机

5、测试用例

在 `python/example_motor_control.py` 中提供了电机控制演示程序，包含三种测试模式：

1. 电机循环控制：控制电机进行正反转运动，并读取电机状态信息。
2. 简单位置控制：进行位置控制，并读取电机状态信息。
3. 获取电机状态：不控制电机，读取电机状态信息，此时电机可外力转动。

二、控制函数

名称解释：

- 电机状态数据：包含位置、速度、力矩、模式、错误码。
 - 模式和错误码需 SDK、通讯板、电机固件版本都在 v4 以上，否则恒为 0。

motor

- 注意事项：

- 统一控制：每次操作控制同一个 CAN 通道上的所有电机。
- 同一个 can 通道的电机控制模式一致：
 - 同一 CAN 通道下所有电机只能使用相同控制模式，
 - 在调用不同模式的函数时，切换模式时会清空缓冲区。
- 批量发送：`motor_send_2()` 一次性发送所有缓冲指令

• 例子：

- 例 1：

控制电机 1、2 为位置模式，设置位置为 1，再控制电机 3 为速度模式，设置速度为 0.1，再使用发送函数 `motor_send_2()`，只有电机 3 响应进行控制，电机 1 和电机 2 数据都将被清空。

代码块

```
1  # 电机1、2设置为位置模式
2  robot.Motors[0].position(1)  # 添加到缓冲区
3  robot.Motors[1].position(1)  # 添加到缓冲区
4
5  # 电机3设置为速度模式（模式切换，将先清楚缓冲区，再写入指令）
6  robot.Motors[2].velocity(0.1)
7
8  # 发送指令（此时只有速度模式指令存在）
9  robot.motor_send_2()  # 仅电机3会响应，电机1、2的指令已被清除
```

- 例 2：

下列控制电机的函数是将控制指令的数据帧存储在数组中，在调用 `motor_send_2()` 函数时统一发送。

代码块

```
1  # 向缓冲区写控制指令
2  robot.Motors[0].position(position)
3  robot.Motors[1].position(position)
4  robot.Motors[2].position(position)
5  robot.Motors[3].position(position)
6  robot.Motors[4].position(position)
7
8
9  # 发送缓冲区的控制指令
10 robot.motor_send_2()
```

1、位置控制模式

说明：

1. 电机将以最大速度和最大加速度运动到指定目标位置，并返回电机状态信息。
2. 参数说明：
 - `position`：目标位置，单位：弧度（rad）。

注意：

1. 该模式下速度与力矩均为最大，运动过程较为激烈。
2. 瞬时电流可能会飙到 5A~10A，若电源响应不够快，或电源电流限制过小，可能导致电机在瞬间获得的电流不足，从而报错。
3. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_send_2()` 才会生效。

代码块

```
1 motor.position(position)
```

2、速度控制模式

说明：

1. 电机以最大加速度加速到指定目标速度，并返回电机状态信息。
2. 参数解析：
 - `velocity`：目标速度，单位：转/秒（rps）。

注意：

1. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_send_2()` 才会生效。

代码块

```
1 motor.velocity(velocity)
```

3、力矩控制模式

说明：

1. 电机按照设定的目标力矩进行转动，并返回电机状态信息。
2. 参数解析：
 - `torque`：目标力矩，单位：牛·米（N·m）。

注意：

1. 函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_send_2()` 才会生效。

2. 若设置的力矩过小，电机可能无法转动。

代码块

```
1 motor.torque(torque)
```

4、电压控制模式

说明：

1. 通过给定的 Q 相电压控制电机运行，并返回电机状态信息。
2. 参数解析：
 - `voltage`：Q 相电压，单位：伏特（V）。

注意：

1. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_send_2()` 才会生效。

代码块

```
1 motor.voltage(voltage)
```

5、电流控制模式

说明：

1. 通过给定的 Q 相电流控制电机运行，并返回电机状态信息。
2. 参数解析：
 - `current`：Q 相电流，单位：安培（A）。

注意：

1. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_send_2()` 才会生效。

代码块

```
1 motor.current(current)
```

6、位置+速度+最大力矩控制模式

说明：

1. 电机以目标速度运动至指定的目标位置，同时限制最大输出力矩，并返回电机状态信息。
2. 参数解析：
 - `position`：目标位置，单位：弧度（rad）。

- `velocity`：目标速度，单位：转/秒（rps）。
- `torque`：最大允许输出力矩，单位：牛·米（N·m）。

注意：

1. 该模式对输出力矩有限制，若设置的最大力矩过小，电机可能无法达到目标速度。
2. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_send_2()` 才会生效。

代码块

```
1 motor.pos_vel_MAXtqe(position, velocity, torque)
```

7、运控模式

说明：

1. 电机输出力矩的计算公式为：
 - 输出力矩 = （目标位置-当前位置） * `kp` + （目标速度-当前速度） * `kd` + `torque`
2. 参数解析：
 - `position`：目标位置，单位：弧度（rad）。
 - `velocity`：目标速度，单位：转/秒（rps）。
 - `torque`：前馈力矩，单位：牛·米（N·m）。

注意：

1. 需要设置合适的 `kp` 和 `kd`，否则控制效果可能较差。
2. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_send_2()` 才会生效。
3. 位置是不断积分得到的，所以在低频控制下会有意外情况。
4. 不推荐使用该函数，推荐使用运控模式 2 `pos_vel_tqe_kp_kd2`。

代码块

```
1 motor.pos_vel_tqe_kp_kd(position, velocity, torque, kp, kd)
```

8、运控模式 2

说明：

1. 电机输出力矩的计算公式为：
 - 输出力矩 = （目标位置-当前位置） * `kp` + （目标速度-当前速度） * `kd` + `torque`
2. 参数解析：

- `position`：目标位置，单位：弧度（rad）。
- `velocity`：目标速度，单位：转/秒（rps）。
- `torque`：前馈力矩，单位：牛·米（N·m）。

注意：

1. 真正的运控模式，和 `pos_vel_tqe_kp_kd` 区别是：
 - `pos_vel_tqe_kp_kd` 的位置是不断积分得到的，在低频控制下容易出现意外情况。
 - `pos_vel_tqe_kp_kd2` 则避免了此问题，控制更稳定。
2. 该需要设置合适的 `kp` 和 `kd`，否则控制效果可能较差。
3. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_send_2()` 才会生效。

代码块

```
1 motor.pos_vel_tqe_kp_kd2(position, velocity, torque, kp, kd)
```

9、位置+速度+pd 控制模式

说明：

1. 电机输出力矩的计算公式为：
 - 输出力矩 = （目标位置-当前位置） * `kp` + （目标速度-当前速度） * `kd`
2. 参数解析：
 - `position`：目标位置，单位：弧度（rad）。
 - `velocity`：目标速度，单位：转/秒（rps）。

注意：

2. 该需要设置合适的 `kp` 和 `kd`，否则控制效果可能较差。
3. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_send_2()` 才会生效。

代码块

```
1 motor.pos_vel_kp_kd(position, velocity, kp, kd)
```

10、位置+速度+加速度控制（梯形控制）模式

说明：

1. 电机按照恒定加速度运动，实现先加速 → 匀速 → 减速的梯形速度控制，同时返回状态信息。
2. 参数解析：

- `position`：目标位置，单位：弧度（rad）。
- `velocity`：目标速度，单位：转/秒（rps）。
- `acc`：加速度，单位：转每秒平方（rps²）。

注意：

1. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_send_2()` 才会生效。

代码块

```
1 motor.pos_vel_acc(position, velocity, acc)
```

Robot

1、停止函数

说明：

1. 全部电机进入停止模式，电机三相都悬空，使电机可以自由转动。

注意：

1. 不会返回电机状态。
2. 调用此函数时会立即发送指令，无需额外调用发送函数 `motor_send_2()`。
3. 会覆盖缓冲区原有数据。

代码块

```
1 robot.set_stop()
```

2、刹车函数

说明：

1. 将电机所有相短接到地，实现“阻尼刹车”效果。
2. 刹车阻力与电机转速成正相关。

注意：

1. 不会返回电机状态。
2. 刹车依靠电机自身阻尼实现，有外力作用时电机仍可缓慢转动。
3. 调用此函数时会立即发送指令，无需额外调用发送函数 `motor_send_2()`。
4. 会覆盖缓冲区原有数据。

```
1 robot.brake()
```

3、重启函数

说明：

1. 对电机执行软件重启。

注意：

1. 使用重启指令后，电机大约需要 50ms 才可正常控制，函数内部延时 100ms，
2. 电机是单圈绝对值编码器，重启后读取电机位置是单圈数据，即在 -1.717~1.717 弧度内。
3. 不会返回电机状态。
4. 调用此函数时会立即发送指令，无需额外调用发送函数 `motor_send_2()`。

代码块

```
1 robot.reset()
```

4、读取状态函数

说明：

1. 发送指令读取电机当前状态，用于在不控制电机情况下读取电机状态信息。

注意：

1. 调用此函数时会立即发送指令，无需额外调用发送函数 `motor_send_2()`。
2. 调用此函数不会对电机进行控制，仅获取状态信息。

代码块

```
1 robot.send_get_motor_state_cmd()
```

5、发送函数

说明：

1. 将缓冲区中已设置的指令数据发送给电机。

注意：

1. 所有在 motor 中的电机控制函数（如位置、速度、力矩等）仅写入缓冲区，不会立即生效，需要调用此函数发送指令才会发送，而在 robot 里的控制函数，内部会自行调用 `motor_send_2`。

代码块

```
1 robot.motor_send_2()
```

电机状态函数

1、获取电机状态函数

说明：

1. 在缓冲区读取电机的 id、控制模式、错误码、位置、速度及力矩信息。

注意：

1. 模式和错误码获取需要 SDK、通讯板、电机固件版本都在 v4 以上才可以读取这两项数据，否则这两项恒为零。
2. 电机状态信息的解析在子线程中完成，无需关注。

代码块

```
1 state = motor.get_current_motor_state()
2         print(f"Motor {state.ID}: mode={state.mode}, "
3               f"fault=0x{state.fault:02X}, "
4               f"pos={state.position:.6f}, "
5               f"vel={state.velocity:.6f}, "
6               f"tor={state.torque:.6f}")
```

三、配置文件

机器人配置文件

主配置文件位于 `robot_param/robot_config.yaml`：

代码块

```
1 robot:
2     name: "HI"
3     param_file: "../robot_param/1dof_STM32H730_model_test_0rin_params.yaml"
```

`1dof_STM32H730_model_test_0rin_params.yaml` 是电机参数文件，在 `robot_param` 目录下有多个电机参数文件可以选择，根据电机使用情况选择文件，并写在 `robot_config.yaml` 的路径上。

yaml 配置文件说明

`rb.Motor` 是一个 `vector<motor*>` 列表，可以理解为一个装载了机器人上所有的电机接口的一个列表，可以通过这个列表去控制机器人上的电机。但是这个列表的顺序是机器人的配置文件决定

的。

- sdk 提供了多个机器人配置文件示例，可根据需要自行新建或修改。
- 以 单个电机为例，编写配置文件如下：

代码块

```
1  robot:
2      robot_name: "Single_Motor"
3      Serial_Type: "/dev/ttyACM"
4      Serial_baudrate: 4000000
5      motor_timeout_ms: 0 # 设置电机的超时时间（单位：毫秒），取值范围：[0, 32760]，
                           如果电机在指定时间内未接收到新的控制指令，将进入阻尼模式。将此值设为0表示禁用超时时间功
                           能。
6      CANboard_num: 1
7      CANboard:
8          No_1_CANboard:
9              CANport_num: 1
10             CANport:
11                 CANport_1:
12                     serial_id: 1
13                     motor_num: 1
14                     motor:
15                         motor1:
16                             type: "4438_30"
17                             id: 1
18                             name: "L_low_foot"
19                             num: 1
20                             pos_limit_enable: false
21                             pos_upper: 5
22                             pos_lower: -5
23                             tor_limit_enable: false
24                             tor_upper: 5
25                             tor_lower: -3
```

根据以上文件，可以把配置文件分为以下 4 个部分：

1. 机器人根节点

- 配置文件以 `robot` 开始。

2. 软硬件参数说明

- `robot_name: "Single_Motor"`：表示机器人的名字为 `Single_Motor`，可自定义。
- `Serial_Type: "/dev/ttyACM"`：表示机器人通讯板的串口名称为 `/dev/ttyACM*`，一般无需改动。

- `Serial_baudrate:4000000`：表示机器人通讯板的串口波特率为 4000000，一般无需改动。
- `motor_timeout_ms: 0`：表示设置超时时间：
 - 当电机在设定时间内未收到新指令时进入阻尼模式，用于防止 can 通信异常电机仍然运动。
 - 默认设置为 0 表示关闭该功能。
- `CANboard_num: 1`：表示机器人使用通讯板数量。

3. 通讯板配置信息

- `CANboard:`：表示机器人的通讯板信息，包含 CAN 通道信息和每个 CAN 通道上的电机信息。
- `No_1_CANboard:`：表示第一块通信板的节点。
- `CANport_num:1`：表示通讯板使用 CAN 通道数量。
- `CANport`：表示 CAN 通道信息，包含 CAN 通道节点和 CAN 通道上的电机信息。
- `CANport_1`：表示第一个 CAN 通道的节点，只是一个名称，要求从 `CANport_1` 开始。
- `serial_id:1`：表示实际使用的 CAN 通道。
- `motor_num: 1`：表示这个 CAN 通道上的电机个数。

4. 电机配置信息

- `motor_1`：表示这个 CAN 通道上的第 1 个电机信息。
- `type: "4438_30"`：电机类型，用于电机输出力矩，填错不影响电机转动，但影响电机输出力矩的准确。可选项有：
 - `3536_32`
 - `4538_19`
 - `5046_20`
 - `5047_09`
 - `5047_36`：这是旧版的5047_36，现已停产，保留是为了兼容旧算法，推荐使用 `5047_36_2`。
 - `4438_30`
 - `4438_32`
 - `6056_36`
 - `5043_20`
 - `7256_35`
 - `60sg_35`

- 60bm_35
- 5047_36_2：相当于新版的5047_36，推荐使用。
- General
- id: 1：电机 ID
- name: "L_low_foot"：电机名称，可自定义。
- pos_limit_enable: false：位置保护使能标识，true 表示使能，false 表示不使能。
- pos_upper: 5：位置上限，单位：弧度（rad）。
- pos_lower: -5：位置下限，单位：弧度（rad）。
- tor_limit_enable: false：力矩限制使能标识，true 表示使能，false 表示不使能。
- tor_upper: 5：力矩上限，单位：牛·米（N·m）。
- tor_lower: -3：力矩下限，单位：牛·米（N·m）。

注意：

- 每个 CAN 通道上的电机 ID 必须从 1 开始。
- 使用时配置文件中的 id 要与使用电机 ID 一致。
- serial_id 要与实际连接的 CAN 通道一致。
- 电机按照配置文件中的顺序进行排序
 - 优先按 CANport 顺序排列（CANport_1 → CANport_2 → ...）
 - 每个 CANport 内按 motor 顺序排列（motor1 → motor2 → ...）
 - 例：
 - 使用了 6dof_*.yaml 配置文件，连接了 12 个电机，为了举例方便，只保留 CANport 和 motor 关系，其他详细配置省略。

代码块

```

1  #配置文件
2  CANport:
3      CANport_1:
4          motor:
5              motor1:...
6              motor2:...
7              motor3:...
8              motor4:...
9              motor5:...
10             motor6:...
```



```
11     CANport_2:
12         motor:
13             motor1:...
14             motor2:...
15             motor3:...
16             motor4:...
17             motor5:...
18             motor6:...
```

- 对 CANport_1 的 motor1、motor2 和 CANport_2 的 motor2、motor3 进行控制电机的操作如下

代码块

```
1  # CANport_1 的6个电机 → 编号0-5
2  # CANport_2 的6个电机 → 编号6-11
3
4  #CANport_1的motor1
5  robot.Motors[0].pos_vel_MAXtqe (...)
6  #CANport_1的motor2
7  robot.Motors[1].pos_vel_MAXtqe (...)
8
9  #CANport_2的motor2
10 robot.Motors[7].pos_vel_MAXtqe (...)
11 #CANport_2的motor3
12 robot.Motors[8].pos_vel_MAXtqe (...)
13
14 # 发送控制命令
15 robot.motor_send_2()
```